

Practice Questions

1. Expose your Day 4 RAG pipeline as POST /chat in FastAPI. The response body must include answer (string) and sources (list of chunk metadata dicts).

The screenshot shows a REST client interface with the following details:

- URL:** 127.0.0.1:8000/docs#/default/chat_chat_post
- Request Body:**

```
{  "query": "Explain AI"}
```
- Buttons:** Execute (highlighted in blue), Clear
- Responses:**
 - Curl:**

```
curl -X 'POST' \  'http://127.0.0.1:8000/chat' \  -H 'accept: application/json' \  -H 'Content-Type: application/json' \  -d '{  "query": "Explain AI"  }'
```
 - Request URL:** http://127.0.0.1:8000/chat
 - Server response:**

Code	Details
200	Response body <pre>{ "answer": "AI systems can perceive their environment, reason about information, learn from past experiences, and take autonomous actions to achieve defined goals.", "sources": [{ "source": "document.txt", "chunk_index": 0 }, { "source": "document.txt", "chunk_index": 1 }, { "source": "document.txt", "chunk_index": 2 }]}</pre>

2. Add conversation history: store the last 6 turns in memory per session and pass them as previous messages to Groq so follow-up questions work correctly.

```
{
  "query": "Explain it more."
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/chat' \
  -H 'accept: application/json' \
  -H 'content-type: application/json' \
  -d '{
    "query": "Explain it more."
  }'
```

Request URL

```
http://127.0.0.1:8000/chat
```

Server response

Code

Details

200

Response body

```
{
  "answer": "AI can be explained in several ways. There's Machine Learning, which allows systems to learn patterns from data and make predictions. It emerged as a solution to the limitations of rule-based AI. Machine Learning can generalize and make accurate predictions on new inputs.\\n\\nThere are also different types of learning, such as Unsupervised learning, which is used for custom or segmentation and anomaly detection. Reinforcement learning is another approach, where an agent learns by interacting with an environment and receiving rewards or penalties.\\n\\nAdditionally, there are systems like Agentic RAG, which can perform complex tasks by selecting tools, observing results, and updating its reasoning. \\n\\nResponsible AI development also requires attention to fairness, transparency, privacy, and safety.",
  "sources": [
    {
      "source": "document.txt",
      "chunk_index": 5
    },
    {
      "source": "document.txt",
      "chunk_index": 7
    }
  ]
}
```

3. Implement a simple cache: hash the query string with hashlib.md5, store results in a dict, and return cached answers immediately on repeated queries. Log whether each request was a cache hit or miss

```
config.py chunk.py ingest.py main.py X cache.py history.py llm.py embed.py
main.py > chat
1 from retrieval import retrieve
2 from prompt import build_prompt
3 from llm import ask
4 from cache import cache, get_cache_key
5 from fastapi import FastAPI
6 from pydantic import BaseModel
7
8 from retrieval import retrieve, retrieve_with_metadata
9 from prompt import build_prompt
10 from llm import ask
11 from history import chat_history, add_to_history
12
13 app=FastAPI()
14 class QueryRequest(BaseModel):
15     query: str
16
17 @app.post("/chat")
18 def chat(request: QueryRequest):
19     query_hash=get_cache_key(request.query)
20     if query_hash in cache:
21         print("Cache hit")
22     else:
```

4. Add streaming using FastAPI's `StreamingResponse` and `stream=True` on the Groq call. Verify tokens appear progressively with a curl command or the browser.

The image shows a VS Code editor with several Python files open: config.py, chunk.py, ingest.py, main.py, cache.py, history.py, llm.py, and request.json. The main.py file is the active editor, showing a FastAPI application with a chat endpoint. The code imports modules for retrieval, prompt building, streaming, caching, and history management. It defines a QueryRequest model and a chat function that checks a cache and retrieves documents from a vector store.

```
from retrieval import retrieve
from prompt import build_prompt
from llm import stream_answer
from cache import cache, get_cache_key
from fastapi import FastAPI
from pydantic import BaseModel
from fastapi.responses import StreamingResponse
from retrieval import retrieve, retrieve_with_metadata
from history import chat_history, add_to_history

app=FastAPI()
class QueryRequest(BaseModel):
    query: str

@app.post("/chat")
def chat(request: QueryRequest):
    query_hash=get_cache_key(request.query)
    if query_hash in cache:
        print("Cache hit")
    else:
        print("Cache miss")
    chunks=retrieve(request.query,"documents")
```

The terminal window at the bottom shows two curl commands being executed in a PowerShell prompt. The first command returns a response from an AI model. The second command returns a response from an AI model.

```
(ai_venv) (base) PS C:\Users\Alina\Desktop\Internship-Artificizen\Week 3\Day 5> curl.exe -N -X POST http://127.0.0.1:8000/chat `
>> -H "Content-Type: application/json" `
>> --data-binary "@request.json"
AI systems can perceive their environment, reason about information, learn from past experiences, and take autonomous actions to achieve defin
ed goals.
(ai_venv) (base) PS C:\Users\Alina\Desktop\Internship-Artificizen\Week 3\Day 5> curl.exe -N -X POST http://127.0.0.1:8000/chat `
>> -H "Content-Type: application/json" `
>> --data-binary "@request.json"
AI
refers
to
systems
that
can
```

5. Write a manual evaluation: prepare 5 question–expected answer pairs about your document. Run each through the RAG pipeline, score each answer as Correct / Partially Correct / Wrong, and calculate a simple accuracy percentage.

```
config.py chunk.py ingest.py main.py evaluation.py cache.py history.py llm.py
evaluation.py > ...
1 from retrieval import retrieve
2 from prompt import build_prompt
3 from llm import ask
4
5 test_cases = [
6     {
7         "question": "What is AI?",
8         "expected": "AI is the simulation of human intelligence."
9     },
10    {
11        "question": "What is Machine Learning?",
12        "expected": "Machine Learning is a subset of AI."
13    },
14    {
15        "question": "What is Deep Learning?",
16        "expected": "Deep Learning uses neural networks."
17    },
18    {
19        "question": "What is Natural Language Processing?",
20        "expected": "NLP enables computers to understand human language."
21    },
22 ]
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

(ai_venv) (base) PS C:\Users\Alina\Desktop\Internship-Artificizen\Week 3\Day 5> & "c:\Users\Alina\Desktop\Internship-Artificizen\Week 3\Day 5\ai_venv\Scripts\python.exe" "c:/Users/Alina/Desktop/Internship-Artificizen/Week 3/Day 5/evaluation.py"

Question:
Who is the CEO of Microsoft?

Expected:
I don't know.

Actual:
I don't know.

Score (C/P/W): c

Accuracy: 80.0 %

6. Install ragas and run a basic evaluation using faithfulness and answer_relevancy metrics on your 5 test cases. Print the scores and identify which question scored lowest and why.

ve defined goals.

Faithfulness: 100.0

Answer Relevancy: 100.0

=====
Question: What are the capabilities of AI systems?

Answer: AI systems can:

1. perceive their environment,
2. reason about information,
3. learn from past experiences, and
4. take autonomous actions to achieve defined goals.

Faithfulness: 100.0

Answer Relevancy: 100.0

=====
Question: What is Machine Learning?

Answer: Machine Learning is a solution to the limitations of rule-based AI, which allows systems to learn patterns directly from data, so that given enough examples, a model can generalize and make accurate predictions on new, unseen inputs.

Faithfulness: 100.0

Answer Relevancy: 95.0

=====
Question: What is Deep Learning?

Answer: Deep learning refers to neural networks with many hidden layers, sometimes numbering in the hundreds or thousands. These deep architectures allow models to learn increasingly abstract representations of data.

Faithfulness: 100.0

Answer Relevancy: 100.0

=====
Question: Who is the CEO of Microsoft?

Answer: I don't know.

Faithfulness: 0.0

Answer Relevancy: 0.0

=====
Average Faithfulness: 80.0

Average Answer Relevancy: 79.0

Lowest Scoring Question:

Who is the CEO of Microsoft?

Reason:

The generated answer was either not fully supported by the retrieved context or was less relevant to the user's question.